

Health Logger — Self-Hosting Guide (Hostinger VPS)

This guide walks you through deploying the Health Logger app on your own Hostinger VPS with your custom domain.

Prerequisites

- **Hostinger VPS** with Ubuntu 22.04+ (or any Debian-based distro)
 - **Domain** pointed to your VPS IP (A record in Hostinger DNS)
 - **SSH access** to your VPS
 - **Node.js 18+** (LTS recommended)
 - **PostgreSQL 14+** (can be on the same VPS or a managed service)
 - **An OpenAI API key** (or compatible provider — see LLM section below)
-

1. VPS Initial Setup

```
# SSH into your VPS
ssh root@YOUR_VPS_IP

# Update system
apt update && apt upgrade -y

# Install Node.js 18 (via NodeSource)
curl -fsSL https://deb.nodesource.com/setup_18.x | bash -
apt install -y nodejs

# Install Yarn
npm install -g yarn

# Install PostgreSQL
apt install -y postgresql postgresql-contrib

# Install Nginx (reverse proxy)
apt install -y nginx

# Install Certbot for HTTPS
apt install -y certbot python3-certbot-nginx

# Install PM2 (process manager)
npm install -g pm2
```

2. Set Up PostgreSQL

```
# Switch to postgres user
sudo -u postgres psql

-- Create database and user
CREATE USER healthlogger WITH PASSWORD 'your_secure_password_here';
CREATE DATABASE healthlogger OWNER healthlogger;
GRANT ALL PRIVILEGES ON DATABASE healthlogger TO healthlogger;
\q
```

Your DATABASE_URL will be:

```
postgresql://healthlogger:your_secure_password_here@localhost:5432/healthlogger
```

3. Deploy the App

```
# Create app directory
mkdir -p /var/www/health-logger
cd /var/www/health-logger

# Extract the package (upload health-logger-selfhost.tar.gz first)
tar xzf /path/to/health-logger-selfhost.tar.gz

# Install dependencies
yarn install

# Generate Prisma client
yarn prisma generate

# Create your .env file
cp .env.example .env
nano .env # Edit with your values (see section 4)

# Push database schema
yarn prisma db push

# (Optional) Seed initial data
yarn tsx scripts/seed.ts

# Build the app
yarn build

# Start with PM2
pm2 start npm --name "health-logger" -- start
pm2 save
pm2 startup # Follow the output command to enable auto-start on reboot
```

4. Environment Variables (.env)

Create a `.env` file with these values:

```
# === DATABASE ===
DATABASE_URL="postgresql://healthlogger:your_secure_password_here@localhost:5432/healthlogger"

# === LLM API (Ollama Cloud  gemma4:31b-cloud with vision) ===
OLLAMA_API_KEY="your-ollama-api-key"

# === PUSH NOTIFICATIONS ===
# Generate new VAPID keys: npx web-push generate-vapid-keys
NEXT_PUBLIC_VAPID_PUBLIC_KEY="your-vapid-public-key"
VAPID_PRIVATE_KEY="your-vapid-private-key"

# === EMAIL (see section 6) ===
SMTP_HOST="smtp.gmail.com"
SMTP_PORT="587"
SMTP_USER="your-email@gmail.com"
SMTP_PASS="your-app-password"
EMAIL_FROM="Health Logger <your-email@gmail.com>"

# === APP URL ===
NEXTAUTH_URL="https://yourdomain.com"
```

5. LLM API — Already Configured for Ollama Cloud

The app is already configured to use **Ollama Cloud** with the `gemma4:31b-cloud` model (which has vision support for image analysis). Both the chat and doctor-summary routes point to `https://ollama.com/v1/chat/completions`.

Just set your `OLLAMA_API_KEY` in the `.env` file and you're good to go.

If you ever want to switch to OpenAI or another provider, change the URL in two files:

- `app/api/chat/route.ts` (search for `ollama.com/v1`)
- `app/api/reports/doctor-summary/route.ts` (same search)

app/layout.tsx

Remove this line (it's a platform-specific script, harmless but unnecessary):

```
<script src="https://apps.abacus.ai/chatllm/appllm-lib.js"></script>
```

6. Email — Already Configured with Nodemailer

The app already uses **Nodemailer** for sending health summary emails via SMTP. No code changes needed — just set these env vars:

```
SMTP_HOST="smtp.gmail.com"           # or your Hostinger mail server
SMTP_PORT="587"                     # 587 for STARTTLS, 465 for SSL
SMTP_USER="your-email@gmail.com"
SMTP_PASS="your-app-password"
EMAIL_FROM="Health Logger <your-email@gmail.com>"
```

Gmail Setup

1. Enable 2-Factor Authentication on your Google Account
2. Go to [App Passwords](https://myaccount.google.com/apppasswords) (https://myaccount.google.com/apppasswords)
3. Create a new app password → use it as `SMTP_PASS`

Hostinger Email Setup

If you set up an email account through your Hostinger domain (e.g., `health@yourdomain.com`):

```
SMTP_HOST="smtp.hostinger.com"
SMTP_PORT="465"
SMTP_USER="health@yourdomain.com"
SMTP_PASS="your-hostinger-email-password"
EMAIL_FROM="Health Logger <health@yourdomain.com>"
```

7. Push Notification Cron Jobs

The app has 3 daily push reminders (8 AM, 1 PM, 8 PM Central). On Abacus these were scheduled tasks. On your VPS, set them up as cron jobs:

```
crontab -e
```

Add these lines (adjust timezone — the VPS should be set to America/Chicago or use TZ):

```
# Morning reminder - 8:00 AM Central
0 8 * * * curl -X POST http://localhost:3000/api/push/send -H "Content-Type: application/json" -d '{"title":"Good Morning! ☀️","body":"Time to log your morning BP and meds."}'

# Midday reminder - 1:00 PM Central
0 13 * * * curl -X POST http://localhost:3000/api/push/send -H "Content-Type: application/json" -d '{"title":"Midday Check-in 🩺","body":"Have you taken your midday medications?"}'

# Evening reminder - 8:00 PM Central
0 20 * * * curl -X POST http://localhost:3000/api/push/send -H "Content-Type: application/json" -d '{"title":"Evening Wrap-up 🌙","body":"Log your evening BP and meds before bed."}'
```

Set your VPS timezone:

```
timedatectl set-timezone America/Chicago
```

8. Nginx Reverse Proxy + HTTPS

```
nano /etc/nginx/sites-available/health-logger
```

Paste:

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}
```

```
# Enable the site
ln -s /etc/nginx/sites-available/health-logger /etc/nginx/sites-enabled/
nginx -t && systemctl reload nginx

# Get SSL certificate
certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

9. Prisma Schema — Fix Output Path

Before running on your VPS, update `prisma/schema.prisma` :

Change:

```
output = "/home/ubuntu/health_logger/nextjs_space/node_modules/.prisma/client"
```

To:

```
output = "../node_modules/.prisma/client"
```

(Or simply remove the `output` line entirely — Prisma defaults to `node_modules/.prisma/client`)

10. next.config.js — Simplify

The `experimental.outputFileTracingRoot` and `NEXT_DIST_DIR / NEXT_OUTPUT_MODE` env vars are Abacus-specific. Simplify `next.config.js` to:

```

/** @type {import('next').NextConfig} */
const nextConfig = {
  eslint: { ignoreDuringBuilds: true },
  typescript: { ignoreBuildErrors: false },
  images: { unoptimized: true },
};

module.exports = nextConfig;

```

Summary of Changes Needed

What	Current (Abacus)	Self-Hosted Replacement
LLM API	<code>ollama.com/v1/chat/completions</code>	Already configured for Ollama Cloud
LLM model	<code>gemma4:31b-cloud</code>	Already configured (vision-capable)
Email sending	Nodemailer + SMTP	Already configured — just add SMTP env vars
Push cron jobs	Abacus scheduled tasks	Linux crontab
Database	Abacus-hosted PostgreSQL	Self-hosted PostgreSQL
VAPID keys	Abacus .env	Generate new: <code>npx web-push generate-vapid-keys</code>
Prisma output path	Hardcoded absolute path	Relative <code>../node_modules/.prisma/client</code>
<code>next.config.js</code>	Abacus build flags	Simplified standard config
Layout script	<code>apllm-lib.js</code>	Remove

Hostinger DNS Setup

In your Hostinger control panel:

1. Go to **Domains** → your domain → **DNS Zone**
2. Add/edit an **A record**: `@` → your VPS IP address
3. Add an **A record**: `www` → your VPS IP address
4. Wait for propagation (usually 5-15 min with Hostinger)

Ongoing Maintenance

```
# View logs
pm2 logs health-logger

# Restart after code changes
cd /var/www/health-logger
yarn build
pm2 restart health-logger

# Database backup (set up as a daily cron)
pg_dump -U healthlogger healthlogger > /backups/healthlogger_$(date +%Y%m%d).sql
```

That's it! 🎉